
Summer of Quant

Advanced Project Statement

What You're Building

In one sentence: you're building a program that looks at market data, figures out whether the market is currently calm, falling, or in a full-blown crisis, and automatically reshuffles a portfolio (between stocks, bonds, and gold) to match.

Most simple portfolios just pick fixed weights once say, 60% stocks and 40% bonds and never change them. That works fine when markets behave normally, but falls apart during a crash, because a fixed split has no way of knowing the world has changed. Your job is to build something smarter: a system that detects when the “mood” of the market shifts, and adjusts the portfolio in response.

To do this you'll combine two things you'll learn through this project: a way to statistically detect hidden market “regimes” from price data (using a Hidden Markov Model, explained below), and a way to mathematically pick the best portfolio weights for whatever regime you're currently in (using convex optimization). The hard and genuinely interesting part isn't the math itself, it's actually making sure your system isn't quietly cheating by peeking into the future when you test it, which is a trap almost everyone falls into the first time.

Two Ideas You Need Before You Start

1. What is a “regime”, and what is an HMM?

A “regime” is just a label for the general state the market is in e.g. Bull (calmly rising), Bear (steadily falling), or Crisis (violent, high-volatility moves). The problem is that no one tells you which regime today is in. You only ever see the output of the regime (the prices), not the regime itself. A Hidden Markov Model (HMM) is a statistical tool built exactly for this kind of problem: it looks at observable data (returns, volatility) and infers the most likely hidden state behind it, plus how likely the market is to switch from one state to another. You don't need to derive the math behind it but you need to know how to feed it the right features and interpret what it gives back.

2. What is lookahead bias, and why does it ruin everything?

This is the single most important idea in the whole project, and the easiest one to accidentally get wrong. Lookahead bias happens when your model, while being tested on “past” data, accidentally gets access to information from the future something it could never have known at that point in real life. A simple example: if you calculate the average

volatility of the entire dataset (including 2024 data) and use that number to describe what “high volatility” meant back in 2016, your model in 2016 is secretly using knowledge from 8 years later. Backtests with this bug almost always look amazing because the model is essentially cheating.

Roadmap: How to Approach This

This is a suggested sequence, not a rigid spec. Each phase builds on the previous one, so don't jump to the HMM before your data pipeline is solid because a regime classifier is only as good as the features you feed it. Treat each phase as a checkpoint: get it working end-to-end and inspect the output before moving on.

Get and Understand Your Data

Goal: A clean, aligned dataset of daily prices for a few assets, plus a sense of what “normal” vs “abnormal” looks like in that data.

1. Pull daily price data for at least 3 assets representing different asset classes like NSE, gold etc using yfinance.
2. Convert prices into returns (the day-over-day percentage change), and plot them. Just look at the data before doing anything clever with it.
3. Pull the Indian VIX (a volatility index) as well since it's a useful, well-known proxy for market fear and a natural input to your regime model later.

Engineer Features That Describe the Market's State

Goal: A handful of numerical signals that actually distinguish a calm market from a stressed one.

1. Build momentum features (is the market trending up or down over the last week / month / quarter?) using rolling windows.
2. Build volatility features (how much is the market swinging recently?) using rolling standard deviation of returns.
3. Look at your features around known stressful periods (e.g. 2020, 2022) and sanity-check: does your volatility feature actually spike when you'd expect it to?

Build the Regime Classifier

Goal: A working HMM that takes your features and outputs a regime label (Bull / Bear / Crisis) for every day in your dataset.

1. Feed your engineered features into an HMM (using the hmmlearn library) and fit it to detect 3 hidden states.
2. Since the HMM just gives you state numbers (0, 1, 2) with no names, figure out which state is which e.g. the state with the highest average volatility is probably Crisis.
3. Plot the price chart with your inferred regimes overlaid as colored bands. This is your first real “wow, it's working” checkpoint and should visually line up with periods you already know were turbulent.

Test It Properly

Goal: Confidence that your regime detection and any results from it would have actually worked in real time, not just in hindsight.

1. Set up a walk-forward validation harness: split your data into a sequence of “train on the past, test on the next chunk” windows, sliding forward through time.
2. Re-fit your HMM separately inside each training window. Any feature scaling (e.g. z-scores) must also be computed using only training data.
3. Confirm there's no leakage: a regime label assigned to a given day should never have been influenced by data from after that day.

Connect Regimes to Portfolio Decisions and Backtest

Goal: A backtested strategy that changes the portfolio's asset weights based on the detected regime, compared fairly against simple static benchmarks.

1. Define what each regime should optimize for e.g. maximize Sharpe ratio in Bull, minimize volatility in Crisis and solve for the optimal weights in each case using cvxpy.
2. Apply a small transaction cost (5–10 basis points) every time the portfolio rebalances, so your strategy isn't secretly trading for free because frequent regime flips can quietly destroy returns once costs are included.
3. Backtest your dynamic strategy against a static 60/40 portfolio and an equal-weight portfolio, using Sharpe ratio, max drawdown, and Calmar ratio as your comparison metrics.

Project Checklist

Use this as a final checklist once you've worked through the roadmap above these are the core capabilities your finished project should demonstrate.

- An HMM-based regime classifier that detects Bull / Bear / Crisis states from NSE data, without manually labelling any days yourself.
- Portfolio weights that shift based on detected regime, different optimization objectives for different regimes.
- A walk-forward validation setup where the HMM is re-fit only on past data at every step
- Transaction costs (5–10 bps per rebalance) explicitly modeled, with results shown both with and without this cost.
- A comparison against static 60/40 and equal-weight portfolios on Sharpe ratio, max drawdown, and Calmar ratio.

What to Submit

- A trained HMM with regime labels plotted on top of historical price data, plus the transition probability matrix (how likely the market is to switch between states).

- Your backtesting code with the walk-forward validation built in with regime charts and equity curve plots included.
- A short performance summary with your Strategy, Sharpe, Sortino, max drawdown, Calmar ratio, and turnover, compared against the static benchmarks.
- One Jupyter Notebook/Python Script that runs the full pipeline top to bottom: data → features → regime detection → optimization → backtest → results.
- A GitHub README explaining your key decisions (e.g. why 3 regimes, why these features), how to run your code, and how to reproduce your results.

TECH STACK

Python 3.9+
hmmlearn
CVXPY
NumPy
Pandas
Matplotlib
yFinance
SciPy

CONCEPTS TO MASTER

Hidden Markov Models & the Viterbi algorithm
Convex portfolio optimization
Walk-forward validation
Sharpe / Sortino / Calmar ratios
Transaction cost (turnover) modelling

DATA SOURCES

Yahoo Finance (yfinance) for multi-asset daily returns (MAIN SOURCE)
FRED API for macro indicators (CPI, yields, spreads)
CBOE VIX historical data